Application and substitution for external higher types

Working note, version 1 — 21 October 2026

Abstract

We extend the substitution calculus of *The Unbearable Notation of Logic* to partial computations. Strict simultaneous evaluation introduces a domain restriction that can invalidate the ordinary substitution equation. Occurrence maps determine this restriction and give an exact formula using only the arguments that occur. We then construct partial function types over partial equivalence relations on program codes. Evaluation descends to their quotients, and the parameter theorem supplies total currying and computable composition of function values. A monad of suspended computations restores the term-substitution equation on computation-valued assignments. Passing these computations as arguments gives a lazy finite-type semantics with substitution and beta equations for arbitrary argument terms. These constructions give a concrete semantic extension of the classical recursion category and specify the realization problem for other recursion categories and their coalgebraic representations.

1 Syntax, values, and computations

The substitution calculus in [1, Sections 3–6 and 8–9] distinguishes a syntactic substitution, an assignment, and evaluation. For a finitary signature Σ, let $T_\Sigma X$ be the set of finite terms with variables in X. A substitution is a total function $\sigma : X \to T_\Sigma Y$. Its extension $\sigma^* : T_\Sigma X \to T_\Sigma Y$ replaces each variable x by the term $\sigma(x)$, simultaneously. If $\tau : Y \to T_\Sigma Z$, their composite is $\tau \star \sigma = \tau^* \sigma$. Structural induction gives $(\tau \star \sigma)^* = \tau^* \sigma^*$, the associativity law for the term monad's Kleisli composition.

For a total Σ-algebra A and an assignment $@ : Y \to A$, write $@t$ for the value of a term. The induced assignment is $(@ \bullet \sigma)(x) = @\sigma(x)$. The substitution law is

$$@(\sigma^* t) = (@ \bullet \sigma)t. \tag{1}$$

To interpret computations, one must specify what an assignment contains and when its entries are evaluated. The distinction between values and computations is the basis of the monadic semantics of [3, 4]. We first determine the domain correction for strict partial evaluation, then construct function types and computation names for which the relevant operations are explicit.

All computations below are deterministic and have nontermination as their only effect. Composition gf means first f, then g. Equality of partial functions includes equality of their domains. For $f : Z \rightharpoonup A$, write $\bar{f} : Z \rightharpoonup Z$ for the identity on $\operatorname{dom} f$. A finite partial tuple $\langle f_1, \ldots, f_k \rangle : Z \rightharpoonup A^k$ is defined exactly where all its components are defined.

2 The domain correction to substitution

Let each n-ary symbol of Σ denote a partial function $f^A : A^n \rightharpoonup A$. Evaluation is strict: it evaluates all immediate subterms before applying f^A. This gives each term t over a finite context X a partial interpretation $[\![t]\!]_X : A^X \rightharpoonup A$. Nullary symbols may also diverge.

Example 2.1. Take $A = \omega$, a constant 0, a divergent constant Ω, and a binary symbol p whose value operation is $p(a, b) = a$. On total assignments, the terms x and $p(x, y)$ denote the same function $A^2 \to A$. Under the substitution $\sigma(x) = 0$, $\sigma(y) = \Omega$, the first term returns 0 and the second diverges. Thus equality on total value assignments is not a congruence for arbitrary computational substitution.

The same issue occurs before any quotient is formed. Evaluating every entry of the substituted assignment produces the nowhere-defined tuple $\langle 0, \Omega \rangle$, even when the outer term is just x.

Proposition 2.2. *Let $t \in T_\Sigma X$ and $\sigma : X \to T_\Sigma Y$, where X, Y are finite. Put*

$$s = \langle [\![\sigma(x)]\!]_Y \rangle_{x \in X}, \qquad g = [\![\sigma^* t]\!]_Y.$$

Then

$$[\![t]\!]_X \, s = g\bar{s}. \tag{2}$$

If $S = \mathrm{FV}(t)$ and $s_S = \langle [\![\sigma(x)]\!]_Y \rangle_{x \in S}$, then

$$[\![t]\!]_S \, s_S = [\![\sigma^* t]\!]_Y. \tag{3}$$

Proof. On $\mathrm{dom}\, s$, every substituted term has a value. Structural induction on t proves that evaluation after substitution and evaluation under this value assignment have the same definedness and value. Outside $\mathrm{dom}\, s$, both sides of (2) are undefined. This proves the first identity.

For the second, apply the first identity in context S. If $\sigma^* t$ converges, every leaf substitution corresponding to an occurrence of a variable in t converges, by strictness. Hence $\mathrm{dom}\, g \subseteq \mathrm{dom}\, s_S$, and $g\bar{s}_S = g$. The empty support case uses the unique total map into the singleton $A^\varnothing$. □

The categorical calculation uses $\pi_x s = [\![\sigma(x)]\!]\bar{s}$ and $\langle g_i e \rangle = \langle g_i \rangle e$ for a nonempty tuple and a restriction idempotent e. For the nullary step, the restriction terminal maps satisfy $!_X s = !_Y \bar{s}$. The same structural induction proves the proposition in a cartesian restriction category, using restriction products and a restriction terminal object for the empty context.

2.1 Occurrence maps and evaluation order

The occurrence construction of [1, Section 9 and Appendix A.5] records a map $o : \{1, \dots, m\} \to \{1, \dots, k\}$: the rth occurrence uses argument $o(r)$. It induces the total routing map $o^* : A^k \to A^m$. For partial maps $f_i : Z \rightharpoonup A$,

$$o^* \langle f_1, \dots, f_k \rangle = \langle f_{o(1)}, \dots, f_{o(m)} \rangle \overline{\langle f_1, \dots, f_k \rangle}. \tag{4}$$

Both sides evaluate the selected coordinates on the intersection of all k component domains. This identity explains why routing after strict tuple evaluation cannot discard an unused divergence.

Let t_{lin} be obtained by giving each variable occurrence of t a fresh variable. With $x_1, \dots, x_k$ the declared variables, the exact occurrence formula is

$$[\![t[\sigma]]\!] = [\![t_{\mathrm{lin}}]\!] \langle [\![\sigma(x_{o(1)})]\!], \dots, [\![\sigma(x_{o(m)})]\!] \rangle. \tag{5}$$

Structural induction proves the equality, including domains. Repeated occurrences evaluate the same deterministic computation and obtain the same value. The fibrewise occurrence maps of [1] record the routing in iterated substitution. These formulas concern algebraic terms with strict operations; abstraction will require the distinction between a function value and a computation producing one.

3 Partial function types on program quotients

Fix an acceptable numbering $(\varphi_e)_{e\in\omega}$ of the unary partial recursive functions. Use the Cantor pairing bijection

$$\langle m,n\rangle = \frac{(m+n)(m+n+1)}{2}+n \quad (m,n\in\omega),$$

whose two projections are computable. The parameter theorem supplies a total recursive function s such that

$$\varphi_{s(e,t)}(a) \simeq \varphi_e(\langle t,a\rangle). \tag{6}$$

Here $\simeq$ means that both sides diverge or both converge to the same number. The code $s(e,t)$ is produced without running the computation on the right.

A partial equivalence relation, or PER, is a symmetric transitive relation $R\subseteq\omega^2$. Its support is $\operatorname{supp} R=\{a : aRa\}$, and its represented set is $|R|=\operatorname{supp} R/R$. We allow arbitrary PERs; their supports and relations need not be computably enumerable. Let $\mathbf{N}$ be equality on ω. Products are coded by pairing: $\langle a,b\rangle\,(R\times S)\,\langle a',b'\rangle$ exactly when aRa' and bSb'.

Definition 3.1. An arrow $f : R \rightharpoonup S$ is a partial function $|R| \rightharpoonup |S|$ with a program e such that, for every $a\in\operatorname{supp} R$,

$$f([a]_R)\downarrow \quad\Longleftrightarrow\quad \varphi_e(a)\downarrow,$$

and, when defined, $\varphi_e(a)\in\operatorname{supp} S$ and $f([a]_R)=[\varphi_e(a)]_S$. Such a program *tracks* f. Write $\mathcal{P}$ for the category of PERs and these arrows, and $\mathcal{V}=\mathrm{Tot}(\mathcal{P})$ for its total subcategory.

The definition requires definedness to be invariant under the input PER. Identity is tracked by the identity program, and program composition tracks composition of arrows. Equality of arrows is equality of the represented partial functions, so composition is associative. Restriction is tracked by running the tracker and returning the original input when it halts. Pairing and projections give restriction products. A PER with support $\{0\}$ gives a restriction terminal object. Thus $\mathcal{P}$ is a cartesian restriction category. Its full subcategory on the positive finite powers of $\mathbf{N}$ is the classical recursion category, up to the chosen numerical codings.

Definition 3.2. For PERs R,S, define $R\Rightarrow_p S$ by declaring $e\,(R\Rightarrow_p S)\,e'$ if, for every aRa', either
(i) both $\varphi_e(a)$ and $\varphi_{e'}(a')$ diverge; or
(ii) both converge and their values are S-related.

Symmetry follows by interchanging a,a'; transitivity follows by using the middle program at a' and the fact that $a'Ra'$. Thus this is a PER. Its support consists exactly of the trackers of arrows $R\rightharpoonup S$, and its quotient identifies precisely those trackers that induce the same arrow. In particular, $\operatorname{supp}(\mathbf{N}\Rightarrow_p\mathbf{N})=\omega$: an arbitrary numerical program denotes a partial numerical function.

Theorem 3.3. *Universal application induces an arrow*

$$\mathrm{ev}_{R,S} : (R\Rightarrow_p S)\times R \rightharpoonup S, \qquad \mathrm{ev}([e],[a])=[\varphi_e(a)]$$

on its domain of convergence. For every PER Q, there is a bijection

$$\mathcal{P}(Q\times R,S) \;\cong\; \mathcal{V}(Q,R\Rightarrow_p S), \qquad h\longmapsto \Lambda h, \tag{7}$$

natural under total changes of parameter Q. Its equations are

$$\mathrm{ev}(\Lambda h\times \mathrm{id}_R)=h, \qquad \Lambda(\mathrm{ev}(g\times\mathrm{id}_R))=g \quad (g\ \textit{total}). \tag{8}$$

Proof. Definition 3.2 makes the evaluator's domain saturated under $(R \Rightarrow_p S) \times R$ and its output class independent of representatives. The universal program tracks it.

If e tracks h, define Λh on names by $t \mapsto s(e,t)$. For tQt' and aRa', the tracking condition for h gives matching definedness and S-related outputs on $\langle t, a\rangle$ and $\langle t', a'\rangle$. Equation (6) therefore gives $s(e,t)\,(R \Rightarrow_p S)\,s(e,t')$. The compiler is total, so it tracks a total arrow $Q \to R \Rightarrow_p S$.

The first equation in (8) follows from (6). Conversely, if g is total, its value at $[t]$ is a class of programs. Uncurrying and currying return a program with the same partial behavior on every valid R-input, and hence the same class. This proves the second equation and uniqueness. For total $u : Q' \to Q$, both $\Lambda(h(u \times \mathrm{id}_R))$ and $(\Lambda h)u$ classify the same family of partial functions, proving naturality. □

3.1 Composition and the partial-parameter boundary

A total recursive compiler c for composition satisfies

$$\varphi_{c(d,e)}(a) \simeq \varphi_d(\varphi_e(a)),$$

where the right side first runs e. It induces a total arrow

$$(S \Rightarrow_p U) \times (R \Rightarrow_p S) \longrightarrow R \Rightarrow_p U.$$

An identity-program code gives the identity function value. These operations satisfy the unit and associativity equations on the quotients. Program indices for different compilation orders need not be equal.

The restriction to total parameters in Theorem 3.3 is exact. Suppose $|Q'|$ is nonempty and $u : Q' \rightharpoonup Q$ is nowhere defined. Then $(\Lambda h)u$ is nowhere defined. But $\Lambda(h(u \times \mathrm{id}_R))$ is total and returns the class of the everywhere-divergent function. A divergent computation of a function value and a function value that diverges on every argument are distinct. The next construction preserves this distinction during substitution.

4 Computation names and the substitution law

For a PER S, define a lifting PER LS on program indices by

$$e\,(LS)\,e' \quad \Longleftrightarrow \quad \begin{cases} \varphi_e(0), \varphi_{e'}(0) \text{ both diverge;} & \text{or} \\ \varphi_e(0), \varphi_{e'}(0) \text{ both converge to } S\text{-related values.} \end{cases}$$

A valid name is a suspended computation that either diverges or returns a valid S-name. The quotient $|LS|$ consists of a bottom element and one element for each member of $|S|$. Naming a computation does not decide whether it will converge.

Proposition 4.1. *The construction L is a monad on $\mathcal{V}$, and there are natural bijections*

$$\mathcal{P}(R,S) \cong \mathcal{V}(R, LS) \tag{9}$$

identifying $\mathcal{P}$ with its Kleisli category.

Proof. For $f : R \rightharpoonup S$ tracked by e, the parameter theorem produces, uniformly and totally in a, a code for the suspended computation $\varphi_e(a)$. This defines a total arrow $\widehat{f} : R \to LS$. Conversely, a total arrow $R \to LS$ can be followed by forcing the returned computation name on input 0. The

definition of LS supplies precisely the invariant definedness and values required of an arrow in $\mathcal{P}$. These constructions are inverse on represented functions.

For total $f : S \to T$, Lf sends a computation name to a new name that runs it and then applies a tracker of f. The unit $\eta_S : S \to LS$ returns a code for the computation immediately returning its input name. The multiplication $\mu_S : LLS \to LS$ returns a code that runs the outer computation and then the resulting inner computation. All these code-producing programs are total on valid inputs, by the parameter theorem. Independence of trackers follows from equality of the represented results and their definedness.

The monad laws follow by comparing executions: returning before or after a computation preserves its convergence and value; either parenthesization of flattening runs the same three successive computations. Kleisli composition runs the first computation and then the second, so (9) preserves identities and composition. □

The monad has computable strength $R \times LS \to L(R \times S)$: retain the R-name and return a suspended computation that forces the S-name and pairs the result with it. This gives the value/computation semantics of [3, 4] concretely on the program quotients. Combining (7) and (9) gives

$$\mathcal{V}(Q \times R, LS) \cong \mathcal{V}(Q, R \Rightarrow_p S).$$

Thus $R \Rightarrow_p S$ represents the function space from values of type R to computations of type S.

4.1 Restoring the paper's equation

Let A be a PER and interpret each symbol f of Σ by an arrow $f : A^n \rightharpoonup A$ of $\mathcal{P}$. Define a total operation $f^L : (LA)^n \to LA$ as follows: from n computation names, construct a suspended computation that forces them in order and then runs a tracker of f on the resulting values. The program constructing this name is total. The represented computation may diverge. For a nullary operation it simply suspends that operation.

These operations give the underlying set $|LA|$ an ordinary total Σ-algebra structure. Assignments now have type $@ : X \to |LA|$: every variable is assigned a computation value, including possibly bottom.

Theorem 4.2. *For this algebra and every syntactic substitution $\sigma : X \to T_\Sigma Y$,*

$$@(\sigma^* t) = (@ \bullet \sigma)t \qquad (t \in T_\Sigma X).$$

Equality is equality of computations: both diverge, or both return the same A-class. If the signature and its primitive trackers are effective, the construction is uniformly computable from a finite term, substitution terms for its variables, and computation names for the finitely many assignment entries used.

Proof. The variable case is the definition of $@ \bullet \sigma$. At a node $f(t_1, \ldots, t_n)$, apply the induction hypotheses to each subterm and then the total operation f^L. This proves the equation by structural induction, exactly as for the total term algebra in [1]. Recursive syntax processing and the total suspension compilers prove the effective assertion. □

An unused divergent computation is now an available assignment entry whose name is never forced. The distinction underlying Example 2.1 is

$$(|A|_\perp)^X \quad \text{versus} \quad (|A|^X)_\perp.$$

The first set records partial information coordinate by coordinate; the second records only success or failure of the whole tuple. The algebra on $|LA|$ distinguishes x from strict $p(x, y)$ and makes equality on all computation-valued assignments a substitution congruence. The equality is relative to this interpretation of the primitive symbols. In particular, strict lifting of the numerical projection $p(a, b) = a$ forces both input computations. It is different from the structural projection $(LA)^2 \to LA$, which simply selects the first computation value. The equation $p(x, y) = x$ of the original value algebra therefore need not hold in the lifted algebra.

5 Higher-type application and strict sequencing

Starting with $A_\iota = \mathbf{N}$, define finite value types by

$$A_{\sigma\times\tau} = A_\sigma \times A_\tau, \qquad A_{\sigma\to\tau} = A_\sigma \Rightarrow_p A_\tau.$$

Computations of type σ are represented by LA_σ. Application is the partial arrow $A_{\sigma\to\tau} \times A_\sigma \rightharpoonup A_\tau$ of Theorem 3.3. Suspending it gives the total value-category arrow

$$A_{\sigma\to\tau} \times A_\sigma \longrightarrow LA_\tau.$$

A body $h : \Gamma \times A_\sigma \rightharpoonup A_\tau$ has a total abstraction $\Lambda h : \Gamma \to A_{\sigma\to\tau}$. These provide partial application and abstraction at every finite type.

Substitution that first computes a value context is strict sequencing. If $u : \Delta \rightharpoonup \Gamma$ and $t : \Gamma \rightharpoonup A$, then the computation $let\ \gamma \leftarrow u\ in\ t$ is tu. Its domain is

$$\{d \in \operatorname{dom} u : u(d) \in \operatorname{dom} t\}.$$

On suspended computations this is monadic bind. For a parameterized body $h : \Gamma \times R \rightharpoonup S$ and an argument $u : \Gamma \rightharpoonup R$, the exact application equation is

$$\mathrm{ev}\langle \Lambda h, u\rangle = h\langle \mathrm{id}_\Gamma, u\rangle. \tag{10}$$

Both sides first require u to converge, and then apply the same partial function to its value. If h ignores its second argument but u diverges, both sides diverge. For a total change of context $v : \Delta \to \Gamma$, Theorem 3.3 gives

$$(\Lambda h)v = \Lambda\big(h(v \times \mathrm{id}_R)\big).$$

Thus total value substitution commutes with abstraction, while substitution that first executes a computation is represented by explicit sequencing.

This gives two distinct associative compositions. Syntactic substitution is Kleisli composition for the term monad T_Σ. Execution and strict sequencing are Kleisli composition for the partiality monad L. Theorem 4.2 connects term substitution to evaluation on computation names; equation (10) describes application after a computation produces an argument value. Their types determine which law applies.

6 Lazy application and substitution

Lazy application passes a computation name to the function without first forcing it. For PERs A, B, its function values therefore have type $(LA) \Rightarrow_p B$, rather than $A \Rightarrow_p B$. The evaluator

$$\mathrm{ev} : ((LA) \Rightarrow_p B) \times LA \rightharpoonup B$$

may return a value even when its second argument represents a divergent computation. A program returning 0 without inspecting its input is an example when $B = \mathbf{N}$.

For a body $h : \Gamma \times LA \rightharpoonup B$ and an argument $u : \Gamma \rightharpoonup A$, suspension gives a total arrow $\widehat{u} : \Gamma \to LA$. The application law is

$$\mathrm{ev}\langle \Lambda h, \widehat{u} \rangle = h\langle \mathrm{id}_\Gamma, \widehat{u} \rangle. \tag{11}$$

Unlike (10), this equation does not first execute u. The function body decides whether to force its computation name.

6.1 A finite-type semantics

Consider types generated by a numerical base type ι and arrows. Define value PERs V_σ and computation PERs C_σ by

$$V_\iota = \mathbf{N}, \qquad C_\sigma = LV_\sigma, \qquad V_{\sigma\to\tau} = C_\sigma \Rightarrow_p V_\tau.$$

For a finite context $\Gamma = (x_1 : \sigma_1, \ldots, x_k : \sigma_k)$, put $E_\Gamma = \prod_{i=1}^k C_{\sigma_i}$, with the terminal PER for the empty context. An environment contains computation values; its formation does not execute the computations they name.

There is a total arrow

$$a_{\sigma,\tau} : C_{\sigma\to\tau} \times C_\sigma \to C_\tau. \tag{12}$$

On input names p, q, its tracker returns a suspended program which runs $\varphi_p(0)$ to obtain a function index e, and then runs $\varphi_e(q)$. It passes q itself to e, without first running $\varphi_q(0)$. The parameter theorem makes the code-producing map total. The definitions of the lifting and function PERs prove independence of representatives. Application forces the function computation; the function controls evaluation of its argument.

Use the simply typed lambda calculus with variables, abstraction, application, numerical constants, and a divergent constant Ω_σ at each type. Interpret a judgement $\Gamma \vdash M : \tau$ by a total arrow

$$[\![M]\!]_\Gamma : E_\Gamma \to C_\tau$$

of $\mathcal{V}$. Variables are projections. A numeral denotes the constant immediately returning that number, and Ω_σ denotes the constant bottom computation. Application is interpreted by

$$[\![MN]\!]_\Gamma = a_{\sigma,\tau}\langle [\![M]\!]_\Gamma, [\![N]\!]_\Gamma \rangle.$$

For a body $\Gamma, x : \sigma \vdash M : \tau$, let $h : E_\Gamma \times C_\sigma \rightharpoonup V_\tau$ be the partial arrow corresponding to $[\![M]\!]_{\Gamma,x}$ under (9). Define

$$[\![\lambda x.M]\!]_\Gamma = \eta_{V_{\sigma\to\tau}} \Lambda h. \tag{13}$$

Thus abstraction immediately returns a function value whose body can later force an argument computation. These clauses construct a tracker uniformly from a finite typed term.

Theorem 6.1 (Lazy substitution and beta)**.** *Let θ assign to every $x_i : \sigma_i$ in Γ a term $\Delta \vdash \theta(x_i) : \sigma_i$. Let $M[\theta]$ denote simultaneous capture-avoiding substitution, and put*

$$s_\theta = \langle [\![\theta(x_1)]\!]_\Delta, \ldots, [\![\theta(x_k)]\!]_\Delta \rangle : E_\Delta \to E_\Gamma.$$

For $\Gamma \vdash M : \tau$,

$$[\![M[\theta]]\!]_\Delta = [\![M]\!]_\Gamma s_\theta. \tag{14}$$

For $\Gamma, x : \sigma \vdash M : \tau$ *and* $\Gamma \vdash N : \sigma$*,*

$$[\![(\lambda x.M)N]\!]_\Gamma = [\![M[N/x]]\!]_\Gamma. \tag{15}$$

No convergence assumption on N *is needed.*

Proof. Every component of s_θ is total: it returns a computation name, including a name for bottom. The variable case of (14) is therefore the projection identity. The constant cases hold because reindexing is total. The application case follows by induction, pairing, and composition with $a_{\sigma,\tau}$.

For abstraction, rename the bound variable to avoid capture. Extension of θ by that variable induces the total map $s_\theta \times \mathrm{id}_{C_\sigma}$. By induction and the naturality of (9), the substituted body corresponds to $h(s_\theta \times \mathrm{id}_{C_\sigma})$. Total-parameter naturality in Theorem 3.3 and (13) give the required abstraction equation.

For beta, put $q = \langle \mathrm{id}_{E_\Gamma}, [\![N]\!]_\Gamma \rangle$. Forcing the immediately returned function in (13) and applying it to the argument name gives

$$a_{\sigma,\tau}\langle \eta_{V_{\sigma\to\tau}} \Lambda h, [\![N]\!]_\Gamma \rangle = \widehat{hq} = [\![M]\!]_{\Gamma,x} q.$$

The substitution equation identifies the last expression with $[\![M[N/x]]\!]_\Gamma$. □

In particular, $(\lambda x.0)\Omega_\sigma$ denotes the computation returning 0. A divergent function computation and an immediately returned divergent function remain different: $\Omega_{\sigma\to\tau}$ diverges, whereas $\lambda x.\Omega_{\sigma\to\tau} x$ returns a function value. The eta equation in Theorem 3.3 applies to total families of function values, as its statement requires.

The same principle extends the algebraic substitution calculus in Section 4. One may interpret a primitive symbol by any specified total arrow $(LA)^n \to LA$, including a non-strict one. The proof of Theorem 4.2 still applies. Such an arrow F extends a given value operation $f : A^n \rightharpoonup A$ when $F\eta_A^n = \widehat{f} : A^n \to LA$. For example, the structural projection selects its first computation without forcing the second and satisfies $p(x, y) = x$ on all assignments. A value operation $A^n \rightharpoonup A$ alone does not determine this behavior on divergent arguments; the operation on computation values specifies it.

6.2 Sharing and occurrence maps

Passing a suspended argument is call-by-name. Call-by-need additionally shares its evaluation: variable occurrences referring to the same binding use a single stored computation, whose result is retained once obtained. For the pure lambda calculus, call-by-name and call-by-need induce the same observational equivalence [6, Theorem 32]. The quotient semantics above records convergence and results. To describe sharing itself, an operational presentation must also record the shared bindings.

The occurrence maps of [1, Section 9] specify which uses belong to the same binding. An occurrence map $o : \{1, \ldots, m\} \to \{1, \ldots, k\}$ can route each occurrence to one of k suspended bindings. An empty fibre causes no demand; several occurrences in one fibre can share a result. The maps supply the incidence data. A call-by-need evaluator must in addition implement the store and its update rule. This distinguishes the substitution theorem from an assertion about the cost of executing substitutions.

7 The remaining representation problem

The construction above provides a semantic category containing the classical recursion category, together with explicit application, abstraction, composition compilers, and substitution laws. The use of arbitrary PERs supplies quotient types beyond the computably enumerable supports considered in the regular exact completion of [2, section "Total function spaces and finite observations"]. For example, the partial numerical function type has all indices as support, while its subspace of total functions has support $\mathrm{TOT} = \{e : \varphi_e \text{ is total}\}$. The constructions belong to the realizability approach to higher-type computability surveyed in [5].

Proposition 7.1. *There is an arrow f of $\mathcal{P}$ for which no least restriction idempotent r satisfies $rf = f$.*

Proof. Choose a non-computably-enumerable set $D \subseteq \omega$ containing 0. Let S have full support, with D as one equivalence class and every $n \notin D$ as a singleton class. The total arrow $f : 1 \to S$ selecting $[0]_S$ is tracked by the constant program. For each $n \notin D$, the partial identity $r_n : S \rightharpoonup S$ undefined only at $[n]_S$ is tracked by a program that diverges on input n and otherwise returns its input. Thus $r_n f = f$.

If a least r existed, it would satisfy $r \leq r_n$ for every $n \notin D$, where restriction idempotents are ordered by inclusion of domains. Its domain would therefore consist exactly of $[0]_S$. Since S has full numerical support, any tracker of r would halt exactly on D, a contradiction. □

Consequently, $\mathcal{P}$ does not admit ranges for all arrows, when ranges are least restriction idempotents supporting an arrow. A construction retaining that structure must choose a smaller family of quotients or a different semantic extension.

For an arbitrary recursion category, the next theorem must provide comparable computation names and total parameterization compilers in a specified semantic extension. A coalgebraic realization must in addition interpret evaluation and these compilers and prove compatibility with the quotient relations and their domains. In particular, the application square must express equality of partial maps, including definedness; agreement only where both sides happen to converge is insufficient. The domain and value conditions are precisely those in the partial-application descent proposition of [2, section "Application on quotient types"].

The classical model supplies concrete operations against which such a realization can be tested. The required assertions are preservation of application (8), total parameter substitution, strict sequencing (10), lazy application and substitution in Theorem 6.1, and the computation-valued term-substitution equation. Establishing these for a family of coalgebraic representations would extend the classification of their underlying recursion categories to a comparison of their higher-type operations.

References

[1] F. Lengyel, *The Unbearable Notation of Logic*, version 11, 2026. `https://github.com/flengyel/The_Unbearable_Notation_Of_Logic`.

[2] F. Lengyel, *An External Finite-Type Completion of Recursion Categories*, version 0.11, 2026. `https://github.com/flengyel/External_Finite_Type_Completion`.

[3] E. Moggi, *Notions of computation and monads*, Information and Computation **93** (1991), 55–92. doi:10.1016/0890-5401(91)90052-4.

[4] A. Bucalo, C. Führmann, and A. Simpson, *An equational notion of lifting monad*, Theoretical Computer Science **294** (2003), 31–60. doi:10.1016/S0304-3975(01)00243-2.

[5] J. R. Longley, *Notions of computability at higher types I*, in *Logic Colloquium 2000*, 2005, pp. 32–142. doi:10.1017/9781316755884.004.

[6] J. Maraist, M. Odersky, and P. Wadler, *The call-by-need lambda calculus*, Journal of Functional Programming **8** (1998), 275–317. doi:10.1017/S0956796898003037.